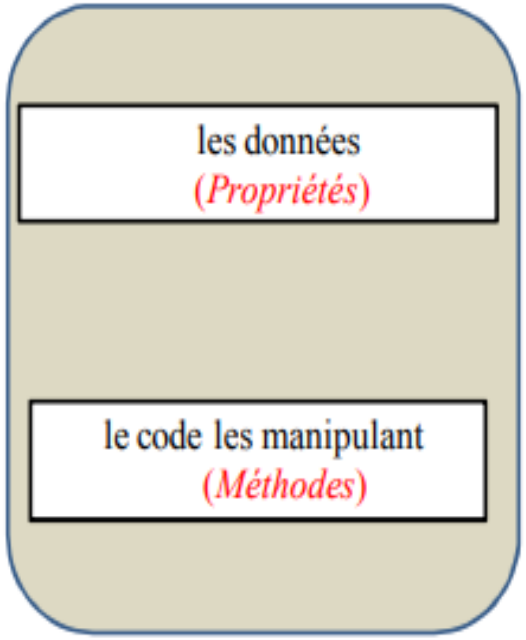
Classe,Objet et Encapsulation

Classe



Classe

- **Les attributs**: représentent la description des données propres à chaque classe d'objets.
- **Les méthodes:** représentent l'ensemble des actions, procédures, fonctions ou opérations que l'on peut associer à une classe.

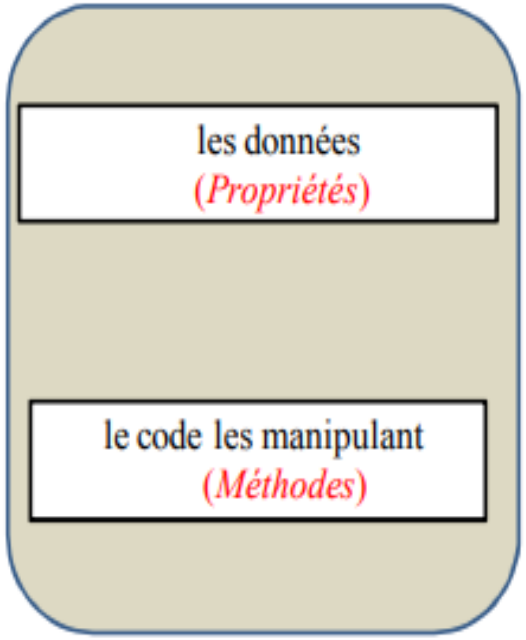
Remarque:

- Le nom de la classe doit commencer **par une majuscule**

Méthode:

- Il devrait commencer par une **lettre minuscule**.
- Il devrait s'agir d'un verbe tel que main(), print(), println().
- Si le nom contient plusieurs mots, commencez par une lettre **minuscule** suivie **d'une lettre majuscule** telle que **actionPerformed()**

Classe



Classe

Variables:

- Il doit commencer par une **lettre minuscule** telle que id, nom.
- Il ne doit pas commencer par les caractères spéciaux comme & (esperluette), $ (dollar), _ (trait de soulignement).
- Si le nom contient plusieurs mots, commencez par la lettre minuscule suivie d'une lettre majuscule telle que firstName, lastName.
- Évitez d'utiliser des variables à un caractère telles que x, y, z.

Constant:

- Il doit être en lettres majuscules telles que ROUGE, JAUNE.
- Si le nom contient plusieurs mots, il doit être séparé par un trait de soulignement(_) tel que MAX_PRIORITY.
- Il peut contenir des chiffres, mais pas comme première lettre.

Exemple: final int **MIN_AGE** = 18;

Identification de l'objet

Chemise
Id prix couleur description quantité dans le stock
ajouter chemise dans le stock diminuer une chemise du stock

Objet 1:
Id: 101
Prix: 20$
Couleur: Bleu
Quantité: 200



Objet 2:
Id: 201
Prix: 30$
Couleur: Rouge
Quantité: 180



objet = instance

Déclaration d'une Classe

```
public class Chemise {

/*Déclaration des attributs*/
/*Déclaration des méthodes*/
//commentaire sur une seule ligne
 /*commentaires sur plusieurs
lignes*/
}
```

➔ Pour créer un objet à partir d'une classe, on utilise l'opérateur **new**



```
public class Test{
 public static void main(String[] args){
Chemise maChemise;
 maChemise = new Chemise();
maChemise.couleur='R';
}
}
```

- L'opérateur **new** fait appel au **constructeur** de la classe
- Un constructeur porte **le même nom** que la classe dans laquelle il est défini
- Un constructeur **n'a pas de type de retour** (même pas void)

Constructeur

❑ Constructeur:

- ➢ est une méthode spéciale qui est appelée lorsqu'un objet est créé.
- ➢ Il existe **deux types de constructeurs**:
  - ❖ **Constructeur par défaut :** est un constructeur qui ne prend aucun paramètre. Si une classe ne définit pas de constructeur explicite, Java fournira automatiquement un constructeur par défaut vide.

    Exemple: **Chemise** maChemise;= **new** Chemise();
  - ❖ **Constructeur surchargé (avec paramètres):** est un constructeur qui prend un ou plusieurs paramètres.

    Exemple: Chemise(int id, char couleur, float prix)

```
{ this.id=id;
  this.couleur=couleur;
  this.prix=prix; }
```

Constructeur

❑ Constructeur:

- ➢ Si le constructeur surchargé est créé, le constructeur par défaut implicite ne sera plus créer par le compilateur.
- ➢ Le compilateur différencie entre les différents constructeurs déclarés au sein d'une même classe en se basant sur **le nombre des paramètres** et **leurs types**.

➔ On ne peut pas avoir deux constructeurs ayant le même nombre et types de paramètres.

```
Chemise(int id) { this.id=id }
Chemise(int id) { this.id=id*2 }
```

Erreur de compilation

➔ Le but d'un constructeur est d'initialiser l'objet avec des valeurs par défaut ou avec les valeurs spécifiées lors de sa création.

Le mot-clé this

❑ this

- ➢ est un mot-clé qui fait référence à l'objet courant dans lequel il est utilisé.
- ➢ L'utilisation de mot clé this:
  - ❖ **Accès à une variable d'instance :**
    - ❑ **Exemple:** Chemise(int id, char couleur) { this.couleur=couleur;}
  - ❖ **Appel d'un autre constructeur:**
    - ❑ **Exemple:** Chemise() { this (40);}
  - ❖ **Passer l'objet courant à une autre méthode :**
    - ❑ **Exemple:** public void afficherDetails() {
      System.out.println(" id: " + this.id);}

Destructeur

❑ Destructeur:

- ❖ Il n'y a pas de destructeur explicite.
- ❖ Il y a des méthodes spéciales appelées "**finalize**" qui peuvent être utilisées pour nettoyer les ressources associées à un objet avant qu'il ne soit détruit par le ramasse-miettes (garbage collector) de Java.
- ❖ La méthode **finalize()** est définie dans la classe Object et peut être remplacée dans une classe dérivée pour spécifier le comportement de nettoyage souhaité.
- ❖ La méthode **finalize()** est appelée automatiquement par le ramasse-miettes de Java lorsqu'il détecte qu'un objet n'est plus accessible par le programme.

Le mot clé static

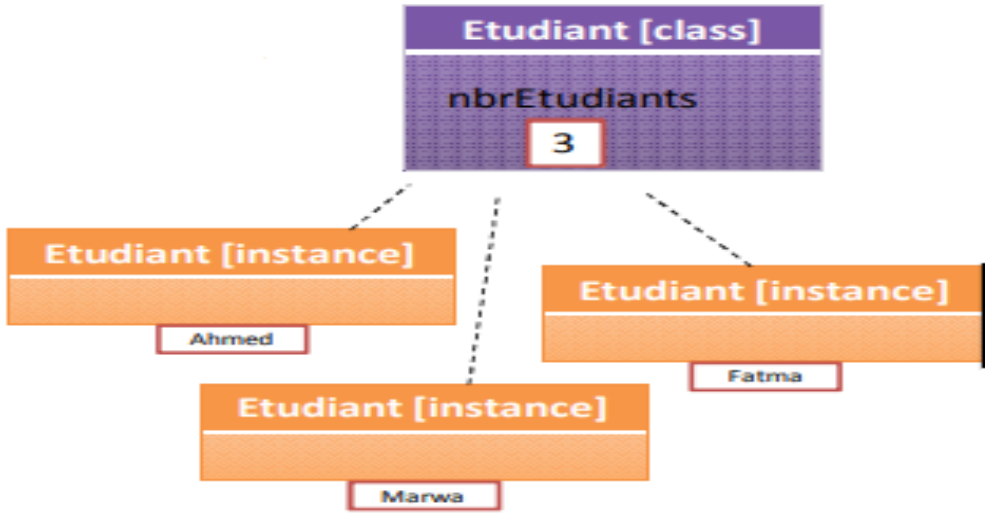
❑ Variable de classe

- ❖ Elle n'appartient pas à une instance particulière, elle appartient à la classe.
- ❖ Elle est partagée par toutes les instances de la classe.
- ❖ **Exemple:**

```
Class Etudiant{
String nom;
static int nbrEtudiants;
Etudiant(String nom){
this.nom=nom;
nbrEtudiants++;}
}
```

→

```
Etudiant etud1 = new Etudiant("Ahmed");
Etudiant etud2 = new Etudiant("Marwa");
Etudiant etud3 = new Etudiant("Fatma");
```



Le mot clé static

❑ Les méthode de classe :

❖ Une méthode statique (ou **méthode de classe**) est une méthode qui appartient à la classe plutôt qu'à une instance spécifique de la classe. ➔ Cela signifie que la méthode **peut être appelée sans avoir besoin d'instancier la classe**, contrairement aux méthodes non statiques qui doivent être appelées sur une instance de la classe.

❖ **Exemple:**

```
class MaClassMath{
 static int min (int a , int b){
if(a<b)
return a;
else
return b;
}
```



Utilisation:

L'appel à une méthode statique se fait en utilisant le nom de la classe.

```
class Test {
public static void main ( String [] args ){
 int x = MaClassMath .min (21 ,4);
} }
```

Le mot clé static

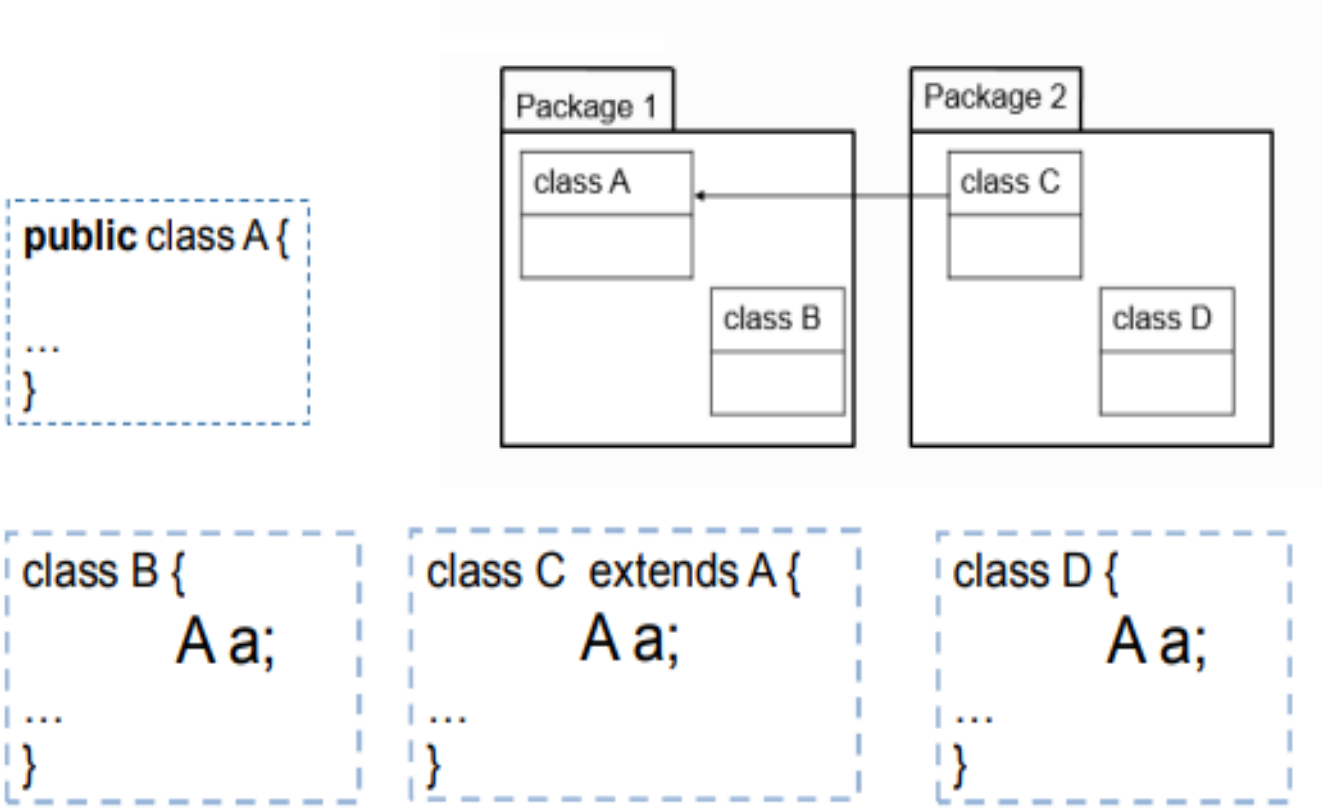
❑ Remarque:

- ❖ Les méthodes d'instances accèdent aux variables d'instance et méthodes d'instances
- ❖ Les méthodes d'instances accèdent aux variables de classe (static) et méthodes de classe (static)
- ❖ Les méthodes de classe (static) accèdent aux variables de classe (static) et méthodes de classe (static)
- ❖ Les méthodes de classe (static) n'accèdent pas aux variables de d'instance et méthodes d'instance
- ❖ Dans une méthode de classe static on ne peut pas utiliser **this**

Encapsulation

❖ Protéger l'information contenue dans un objet.

❑ **Classe public:**

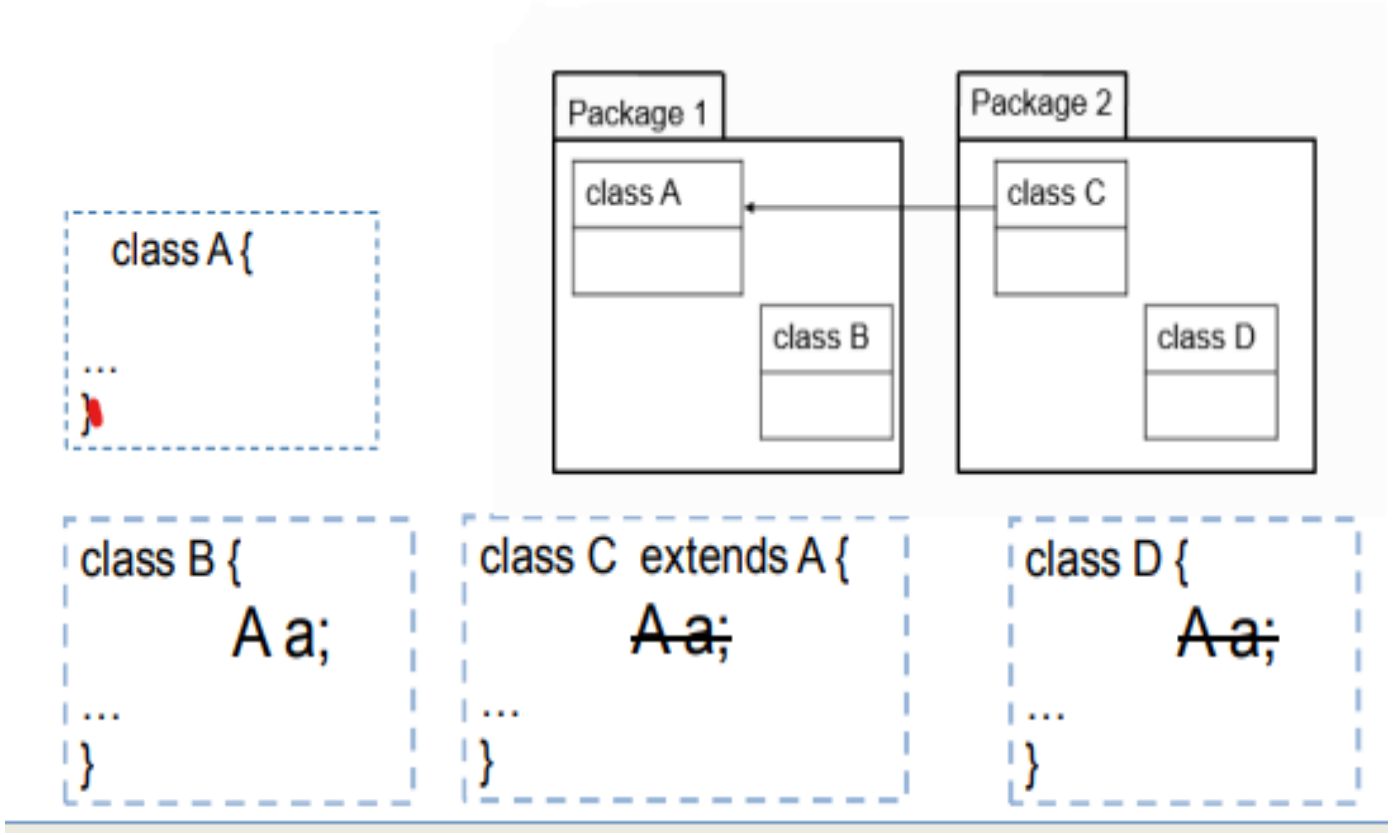


➔ La classe "**public**" est visible depuis n'importe quelle classe du projet.

Encapsulation

❖ Protéger l'information contenue dans un objet.

❑ **Classe par défaut:**



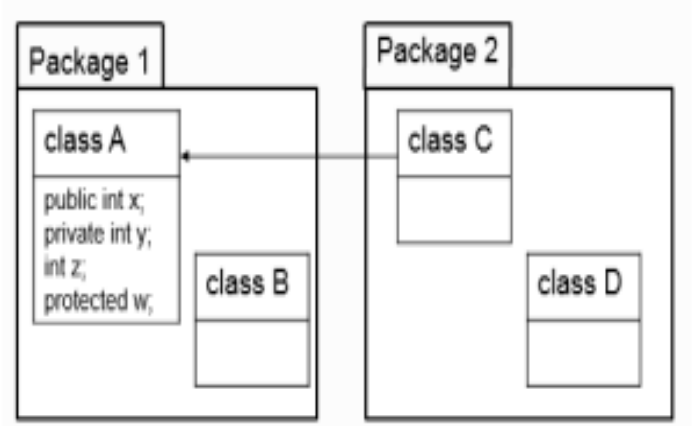
➔ La classe "**par défaut**" est visible seulement par les classes de son package.

Encapsulation

❖ Protéger l'information contenue dans un objet.

❑ L'attribut par défaut:

```
public class A {
   int z ;
...
}
```




```
class B {
 A a=new A();
  a.z= t  ;
}
```

```
class C  extends A {
    A a=new A();
    a. z = t ;
    z = t;
}
```

```
class D {
    A a=new A();
    a. z = t ;
}
```

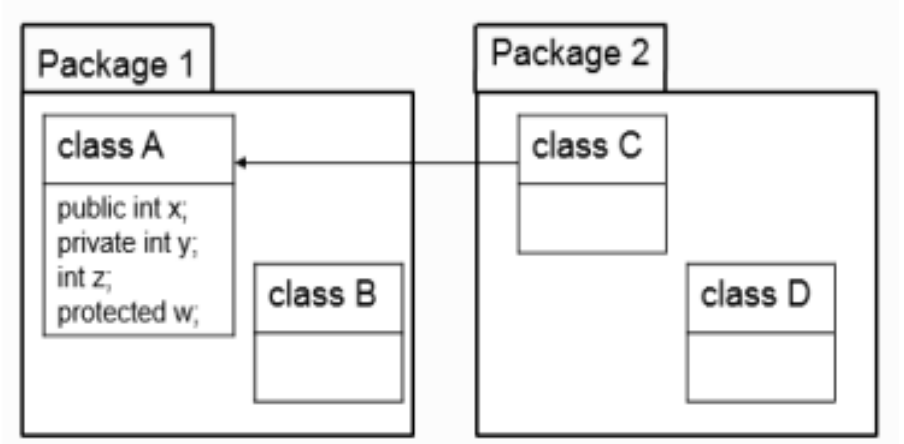
➔ L'attribut "**par défaut**" n'est accessible que depuis les classes faisant partie du même package.

Encapsulation

❖ Protéger l'information contenue dans un objet.

❑ **L'attribut privé:**

```
public class A {
private int y;
…
}
```




```
class B {
 A a=new A();
 a. y = t ;
}
```

```
class C  extends A {
    A a=new A();
    a. y = t ;
    y= t;
}
```

```
class D {
    A a=new A();
    a. y = t ;
}
```

➔ La variable "**private**" n'est accessible que depuis l'intérieur même de la classe.

Encapsulation

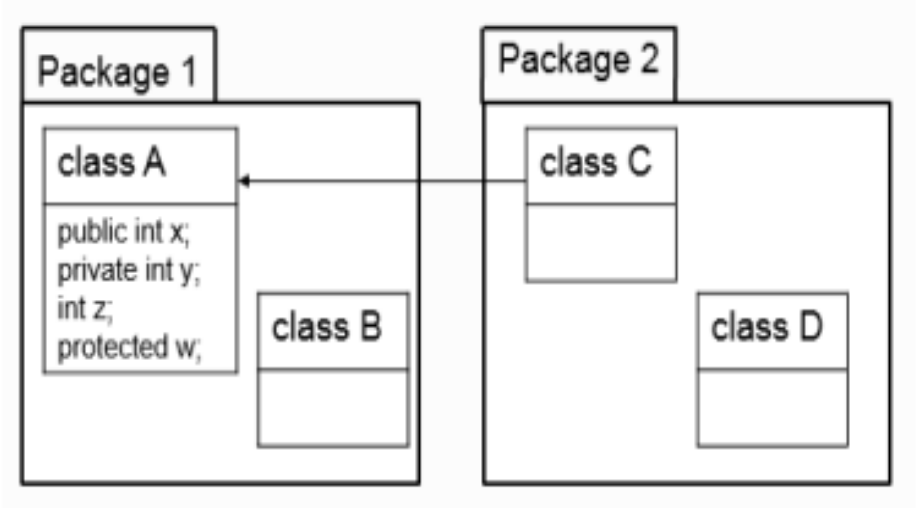
❖ Protéger l'information contenue dans un objet.

❑ L'attribut protégé:

```
public class A {

...
protected int w ;

}
```




```
class B {
 A a=new A();
 a.w= t ;
}
```

```
class C  extends A {
    A a=new A();
    a. w = t ;
    w = t;
}
```

```
class D {
    A a=new A();
    a. z = t ;
}
```

➔ La variable protected est accessible uniquement aux classes d'un package et à ses sous-classes (même si elles sont définies dans un package différent.)

Encapsulation

❖ Pour la manipulation des **attributs private**, il faut :

- **Un mutateur** (**setter)**: est une méthode qui permet de définir la valeur d'une variable particulière
- **Un accesseur** (**getter**): est une méthode qui permet d'obtenir la valeur d'une variable particulière.

➔ Les mutateurs et les accesseurs doivent être déclarés **public**

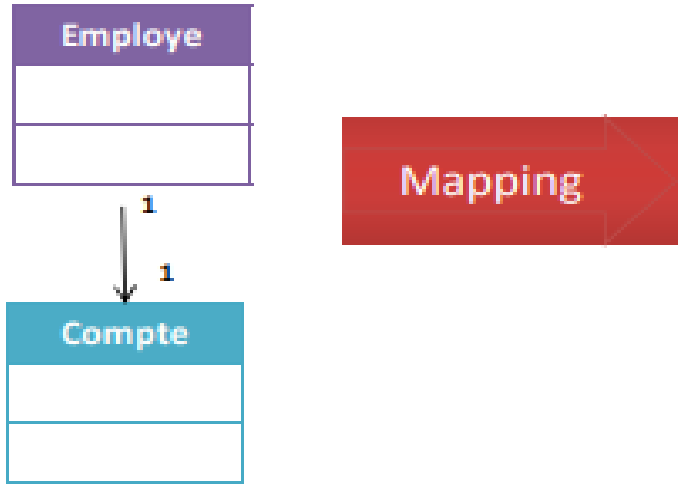
Exemples:

```
private float prix;
public void setPrix(float prix)
{ this.prix=prix; }
 public float getPrix()
{ return prix; }
```

```
private boolean absent;
public void setAbsent(boolean
absent)
{ this.absent=absent; }
public boolean isAbsent()
{ return absent; }
```

Les associations

❑ Association one-to-one undirectionnelle:



Mapping

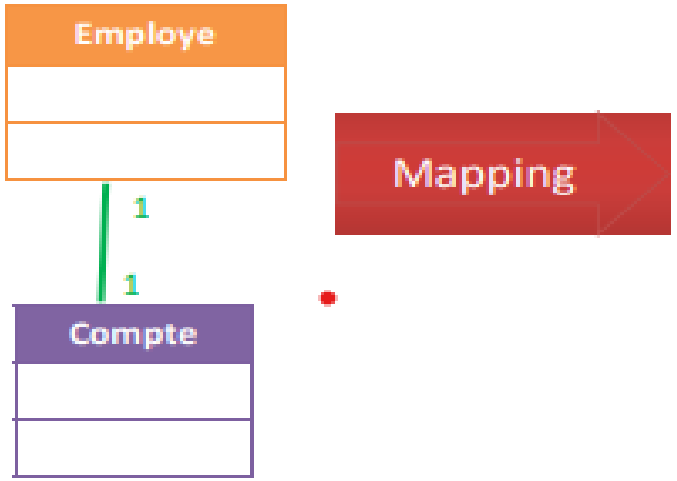
```
class Employe{
private Compte compte;
public void setCompte(Compte compte) {
this.compte=compte; }
public Compte getCompte() {
return compte; } }


  class Compte{


  }
```

Les associations

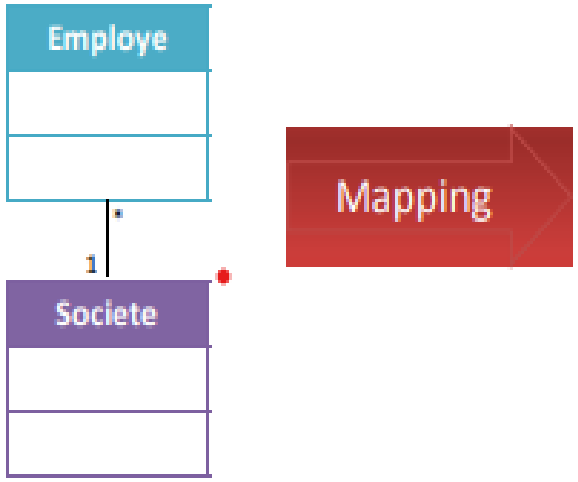
❑ Association one-to-one bidirectionnelle:




```
class Employe{
private Compte compte;
public void setCompte(Compte compte) {
this.compte=compte; }
public Compte getCompte() {
return compte; }
}
class Compte{
private Employe employe;
public void setEmploye(Employe employe) {
this.employe=employe; }
public Employe getEmploye()
 { return empl;oye }
}
```

Les associations

❑ Association one-to-many:



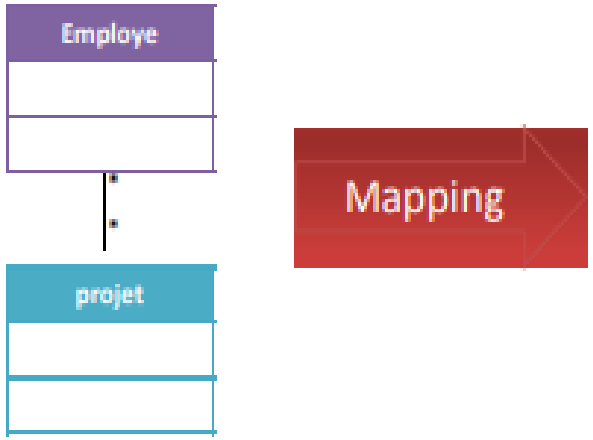
Mapping

```
class Employe{
private Societe societe
public void setSociete(Societe societe)
{ this.societe=societe; }
public Compte getSociete() {
 return societe; } }

class Societe{
private Employe[] employes;
public void setEmployes(Employe[] employes)
{ this.employe=employe; }
public Employe[] getEmployes()
{ return employes } }
```

Les associations

❑ Association many-to-many




```
class Employe{
 private Projet[] projets;
public void setProjets(Projet[] projets) {
this.projets=projets; }
public Projet[] getProjets() { return projets;
} }
```

```
class Projets{
private Employe[] employes;
public void setEmployes(Employe[] employes) {
this.employe=employe; }
public Employe[] getEmployes()
{ return employes } }
```